

# Diseño y Análisis de Algoritmos - INFO145

## Guía de Ejercicios - Unidad II.

Académico: Héctor Ferrada

### 1. Codificación de Huffman y Entropía

En los textos  $T_1$  y  $T_2$ , sólo pueden estar presente los siguientes símbolos:  $\{a, b, c, d, e, f, g, h, i\}$ ,  $T_1$  contiene 1000 símbolos en total y  $T_2$  es de largo 10000. Las probabilidades de ocurrencia para cada símbolo en cada caso son:

$$P_1 = \{21, 13, 5, 19, 11, 4, 12, 5, 10\}$$

$$P_2 = \{9, 0, 29, 9, 4, 6, 12, 5, 26\}, \text{ (el símbolo } b \text{ no esta presente)}$$

En base a lo anterior realice:

1. Determine la cantidad de bits necesarios para codificar en binario cada texto, considerando que todos los códigos deben ser de largos idénticos.
2. Construya el árbol de Huffman para cada caso (dibuje).
3. Implemente (pseudocódigo) la construcción del árbol de Huffman, considerando usar un min-heap para las búsquedas de la menor probabilidad y determine la complejidad de su algoritmo. ¿Cree que es eficiente utilizar un min-heap en la construcción?, justifique.
4. Explique al menos dos utilidades que se le da al árbol de Huffman.
5. En un escenario general, ¿Para que es usada la Entropía y cuando se da su valor máximo?.
6. Utilice el árbol construido para codificar y determine el número de bits necesarios para representar cada texto. Compare los resultados contra la codificación ordinaria y el óptimo, considerando que la entropía en cada caso es:  $H(P_1) \simeq 2,611$  y  $H(P_2) \simeq 2,399$ .

## 2. Programación Dinámica

1. Sea la solución de un problema definida por la siguiente ecuación de recurrencia:  
$$\text{Val}(0) = 1$$
$$\text{Val}(n) = \text{Val}(n-1) + 1 + \text{Val}(\lfloor n/2 \rfloor)$$
Construya la solución inmediata que es un breve algoritmo recursivo y exprese la ec. de recurrencia de su tiempo asintótico. Compare este tiempo con Fibonacci y determine si son diferentes.  
Ahora, ¿Si dispone de un arreglo de tamaño  $n/2$  podrá entregar una solución en tiempo lineal?. De el pseudocódigo de este algoritmo.  
Luego haga un pequeño esbozo del árbol de recurrencia de ambas soluciones.
2. Dado un string  $S = s_1 s_2 \dots s_n$  se desea determinar el mínimo número de símbolos que se deben insertar para convertir  $S$  en palíndromo. Por ejemplo: ACTCG, puede transformarse en AGCTCGA o en GACTCAG con dos inserciones que añaden una 'A' y una 'G'.
3. Suponga la siguiente variante al problema del Rod Cutting. Un cliente desea comprar  $M$  pulgadas de cilindro, para un  $M$  arbitrario que podría llegar a ser incluso mucho más grande que  $n$  — recuerde que  $n$  es el largo de los cortes de cilindro que vende (la tabla  $P$ ). Bien, el asunto con este cliente es que el dinero no es problema, sino que el desea llevarse la menor cantidad de cortes posibles a fin de que la suma de todos los cortes sea igual a  $M$ . Resuelva este problema con PD. Asuma que no necesariamente el vendedor ofrece un corte para una cantidad arbitraria de pulgadas; por ejemplo puede que el largo de las barras sea  $n = 100$  y el vendedor solo ofrezca vender barras completas o de largo 25. Por tanto si el cliente desea comprar en total  $M = 80$  pulgadas de cilindro, entonces no habrá solución y su algoritmo deberá indicar esta imposibilidad.
4. Recuerde el problema de la mochila estudiado en clases. Se tiene una mochila de capacidad  $Q$  y  $n$  objetos  $o_i$ , con  $1 \leq i \leq n$ , los cuales poseen un peso  $w_i$  y un beneficio o valor  $b_i$ ,  $\forall 1 \leq i \leq n$ . Se desea maximizar el beneficio total de los objetos que se pueden guardar en la mochila  $\sum_{i=1}^n (b_i \cdot x_i)$ , validando que  $\sum_{i=1}^n (w_i \cdot x_i) \leq Q$   
Luego llene la matriz  $M[0 \dots n][0 \dots Q]$  para los siguientes objetos de entrada, y compruebe que, para  $n = 5$  y  $Q = 10$ , se obtiene  $M[n, Q] = 130$ :

| Objeto | Peso | Beneficio |
|--------|------|-----------|
| $o_1$  | 1    | 10        |
| $o_2$  | 2    | 40        |
| $o_3$  | 3    | 20        |
| $o_4$  | 4    | 20        |
| $o_5$  | 5    | 60        |

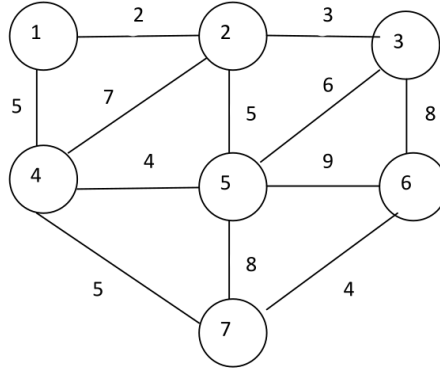
Trabaje ahora en resolver las siguientes variantes:

- a) Suponga que cada objeto  $o_i$  tiene  $c_i$  copias, con  $1 \leq i \leq n$ ; tal que las desciciones ahora no son  $\in \{0, 1\}$ , sino que cada  $x_i \in \{0, 1, \dots, c_i \leq n\}$ .
- b) **Fractional knapsack**. Ahora los elementos pueden fraccionarse; así, cada  $x_i \in \mathbb{R}$ , con  $0 \leq x_i \leq 1$ . Si es necesario, puede asumir que el peso de la fracción de cada objeto sigue siendo un valor entero.

### 3. Algoritmos Greedy

1. Modifique el algoritmo de Kruskal para que en un tiempo de  $O(|E| \log |V|)$  determine un MST, no utilice más de un arreglo auxiliar de tamaño  $|V|$ ; luego aplique el algoritmo y determine un MST para el grafo de la siguiente figura:

| Edges   | (1,2) | (2,3) | (4,5) | (6,7) | (1,4) | (2,5) | (4,7) | (3,5) | (2,4) | (3,6) | (5,7) | (5,6) |
|---------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| weights | 2     | 3     | 4     | 4     | 5     | 5     | 5     | 6     | 7     | 8     | 8     | 9     |

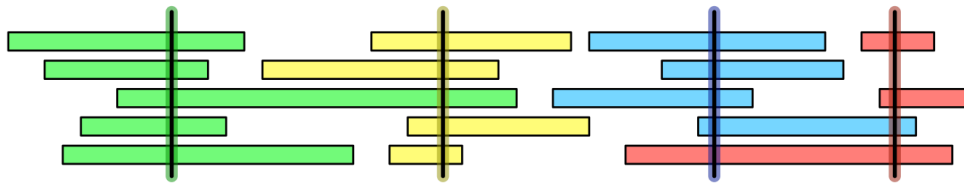


#### 2. Problema 2, Control 2, 2022.

Sea  $X[1 \dots n]$  un conjunto de intervalos cualesquiera en  $\mathbb{R}$ , representados como pares ordenados  $(ini, fin)$  para establecer los puntos de inicio y de termino de cada intervalo. Se define un *interval-cover*  $\mathcal{I}$  para  $X$ , si  $\mathcal{I} \subseteq X$  y la unión de los intervalos en  $\mathcal{I}$  es igual a la unión de los intervalos en  $X$ . Por ejemplo, la siguiente figura resalta a un *interval-cover* para  $X$ .



- i- Entregue un pseudocódigo de un algoritmo exhaustivo que encuentre todos los *interval-cover*'s para  $X$ .
  - ii- Entregue el pseudocódigo de un algoritmo greedy que encuentre un *interval-cover*  $\mathcal{I}$  para  $X$  de tamaño mínimo; es decir que  $|\mathcal{I}|$  sea el menor entre todos los posibles *interval-cover*'s para  $X$ . Entregue también el tiempo asintótico de su algoritmo.
  - iii- De un argumento sólido y breve, que justifique que su algoritmo greedy es correcto.
3. Sea  $X[1 \dots n]$  un conjunto de intervalos cualesquiera en  $\mathbb{R}$ , representados como pares ordenados  $(ini, fin)$  para establecer los puntos de inicio y de termino de cada intervalo. Decimos que un conjunto  $[1 \dots m]$  de puntos en  $\mathbb{R}$  *apuntala* totalmente a  $X$ , si cada intervalo en  $X$  contiene al menos un punto de  $P$ . Por ejemplo para la siguiente figura,  $P$  contiene  $m = 4$  puntos (representado como líneas verticales) y *apuntala* totalmente a  $X$ .



- i- Entregue un pseudocódigo de un algoritmo exhaustivo que determina si un conjunto de puntos  $P[1 \dots m]$  en  $\mathbb{R}$  *apuntala* totalmente a un conjunto  $X[1 \dots n]$  de intervalos  $(ini, fin)$ .
- ii- Entregue el pseudocódigo de un algoritmo greedy que determine un conjunto de puntos  $P$  que *apuntala* totalmente a  $X[1 \dots n]$ . Entregue también el tiempo asintótico de su algoritmo.
- iii- De un argumento sólido y breve, que justifique que su algoritmo greedy es correcto.